INSPIRE BRANDS · QUALITY ENGINEERING

GitHub Copilot for Automation Frameworks

From writing test code to building intelligent, self-documenting frameworks — how AI-assisted development changes the game for QE teams.

Agent Mode | Custom Agents | Selenium + TestNG | Inspire Franchises Framework

🚀 Why GitHub Copilot for Automation Frameworks?

⏱️ Accelerate Development

- Generate Page Object classes from a URL in seconds
- Auto-complete @FindBy locators using site context
- Scaffold new brand test classes following existing patterns
- Reduce boilerplate from hours to minutes

🧠 Understand the Entire Codebase

- Ask questions about any class without reading it
- Navigate complex inheritance chains instantly
- Explain why a test is failing with context-aware analysis
- Onboard new team members in minutes, not days

🔒 Keeps Code Inside Your Org

- Runs inside VS Code — code never leaves your machine
- Respects corporate network and security policies

- GitHub Enterprise support for air-gapped environments
- No external API calls with your proprietary test code

📐 Enforces Team Standards

- Custom agents encode your specific project rules
- Consistent patterns across all developers and brands
- Prevents re-discovery of already-solved problems
- Hard-won fixes become permanent team knowledge

GitHub Copilot vs Cursor IDE

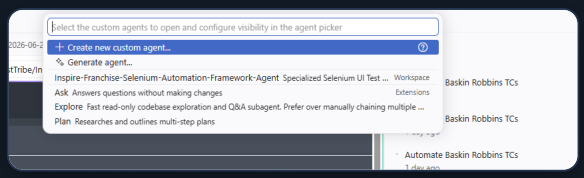
For Enterprise QE Teams

Dimension	GitHub Copilot	Cursor IDE
IDE Integration	**✓ Runs inside VS Code — no context switch**	Separate fork of VS Code
Enterprise Security	**✓ GitHub Enterprise + SSO + audit logs**	Business plan required; limited enterprise controls
Custom Agents	**✓ .agent.md files committed to repo**	Rules files, no agent concept
Team Knowledge Sharing	**✓ Agent file in Git — everyone uses same rules**	Settings per developer, hard to standardize
Automation-Specific Tools	**✓ execute/runTests, execute/testFailure, browser tools**	Generic code tools only
Cost (per developer)	$19/mo (Business) — bundled with GitHub	$20/mo (Business) — separate subscription

Dimension	GitHub Copilot	Cursor IDE
Licensing Model	✓ Already in GitHub org — no new vendor	New vendor relationship required

GitHub Copilot — Three Interaction Modes

Ctrl+Shift+I to open



VS Code — Copilot mode picker (@ shortcut)

ASK

Ask Mode

- Conversational Q&A about your code
- Explains classes, methods, errors
- Answers without making any changes
- Best for: understanding, exploring, learning

"What does AbstractBrandPage do?"

PLAN

Plan Mode

- Researches and outlines multi-step tasks
- Shows what it would do before acting
- Ideal for reviewing changes before applying
- Best for: complex refactoring, new features

"Plan how to add Baskin Robbins brand"

AGENT

Agent Mode

- Autonomously reads, edits, and runs code

- Uses tools: terminal, browser, file system
- Iterates until the task is complete
- Best for: end-to-end automation tasks

"Add Baskin Robbins brand to the framework"

Tip: Start with Ask to understand, use Plan to review scope, use Agent to execute. For automation frameworks — Agent Mode + Custom Agent is the most powerful combination.

ASK

Ask Mode

- Conversational Q&A about your code
- Explains classes, methods, errors
- Answers without making any changes
- Best for: understanding, exploring, learning

"What does AbstractBrandPage do?"

PLAN

Plan Mode

- Researches and outlines multi-step tasks
- Shows what it would do before acting
- Ideal for reviewing changes before applying
- Best for: complex refactoring, new features

"Plan how to add Baskin Robbins brand"

AGENT

Agent Mode

- Autonomously reads, edits, and runs code

- Uses tools: terminal, browser, file system
- Iterates until the task is complete
- Best for: end-to-end automation tasks

"Add Baskin Robbins brand to the framework"

Tip: Start with Ask to understand, use Plan to review scope, use Agent to execute. For automation frameworks — Agent Mode + Custom Agent is the most powerful combination.

Deep Dive — Agent Mode

How Agent Mode Works

- Receives your task in natural language
- Reads relevant files in your workspace
- Makes edits, runs terminal commands, runs tests
- Reads output and iterates if something fails
- Continues until the task succeeds

Tools Agent Mode Can Use

- File system — read, create, edit, rename files
- Terminal — run Maven, compile, execute tests
- Browser — open pages, take screenshots
- Search — semantic search across codebase
- Test runner — run and read test results

What Makes It Different

- Not just code completion — it acts

- Multi-step reasoning across many files
- Self-correcting: reads errors and fixes them
- Combines reading + editing + executing in one loop
- Works with your existing project structure

🎯 **Real Example — Our Framework**

"Add Dunkin' brand to the framework"

Agent automatically:

- Creates DunkinPage.java with correct @FindBy
- Adds case DUNKIN: in BrandPageFactory
- Creates DunkinTest.java extending AbstractBrandTest
- Creates testng-dunkin.xml
- Runs smoke tests to verify ✅

How to Create a Custom GitHub Copilot Agent

1 **Open Agent Picker**
Press @ or
click Agent button
→ "Configure Custom Agents..."

2 **Create New Agent**
Click
"+ Create new custom agent..."
in the picker

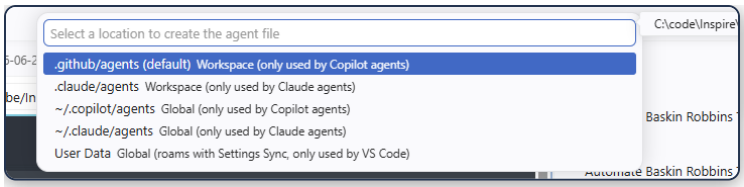
3 **Choose Location**
.github/agents/ (default)
Workspace — shared via Git
with all team members

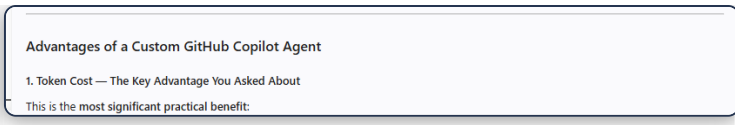
4 **Fill the Template**
Edit the .agent.md
file with YAML front-
matter + instructions

5 **Commit & Share**
git commit & push
All teammates get
the agent instantly

Step 2 — Configure Custom Agents dialog

Step 3 — Choose where to save the agent file





Step 4 — The .agent.md template file opens in VS Code

Section	What it contains
YAML frontmatter	All available tools declared; description for VS Code agent picker
Mandatory rules	clean test always, cookie banner never removed, XPath 5 rules
Project structure	Complete file tree with purpose of every file
Brand table	All 7 brands with enum, URL slug, properties file
Test case inventory	All 34 TCs (TC-H, TC-B, TC-A) with group tags
Design patterns	All SOLID principles + patterns mapped to exact classes
Add a new brand	4 concrete steps with working code — zero guessing
Maven commands	Every combination (headless, groups, browser, profiles)
Reports	Where to find them, how screenshots work
Config reference	Property keys, defaults, system property priority order
Quick class reference	Every class, package, and single-line responsibility
10 documented pitfalls	P-01 to P-10 — every hard-won bug from this session with root cause + fix
TestNG groups	Which TCs belong to which group
Prerequisites	What to install vs what's automatic
Agent behaviour rules	10 enforced rules — prevents re-learning the same mistakes

1
Open Agent Picker
Press @ or
click Agent button
→ "Configure Custom Agents..."

2
Create New Agent
Click
"+ Create new custom agent..."
in the picker

3
Choose Location
.github/agents/ (default)
Workspace — shared via Git
with all team members

4
Fill the Template
Edit the .agent.md
file with YAML front-
matter + instructions

5
Commit & Share
git commit & push
All teammates get
the agent instantly

Speaker notes

Agent File Template

```
---
name: My-Agent
description: What this agent does
tools: ['vscode', 'execute', 'edit', 'search']
---

# Agent instructions go here
```

Agent Scope Options

- .github/agents/ — Workspace only (shared via Git) ← use this
- .claude/agents/ — Workspace (Claude agents only)
- ~/.copilot/agents/ — Global (all workspaces)

```
Define behavior, rules, patterns,
and domain-specific knowledge.
```

- User Data — Global, roams with Settings Sync

Necessary Components of a Custom Agent File

📌 YAML Frontmatter (Required)

- name — agent identifier in picker
- description — shown in the agent list
- tools — explicit tool permissions list
- argument-hint — prompt hint for users

🗺️ Project Structure

- Full file tree with inline annotations
- Tells agent what each file is responsible for
- Prevents wrong-file edits and hallucinations

🔧 Tools Permissions

- Declare exactly what the agent can do
- Groups: vscode, execute, read, edit, search, web
- Principle of least privilege — only what's needed
- Users see permissions before running the agent

🦠 Pitfalls & Solutions

- Documents every hard-won bug with root cause
- Prevents team members re-discovering the same issues
- P-01 to P-N numbered for easy reference

⚠️ Mandatory Rules Section

- NEVER-break rules in ALL-CAPS headers
- Project-specific constraints enforced on every call
- Examples: "always use clean test", "never use text()"

🚦 Agent Behaviour Rules

- Numbered constraints for every code generation action
- Examples: XPath rules, screenshot path rules, brand extension rules
- Ensures consistency across all developers

Tools in the Inspire Agent — What Each Enables

🟦 vscode/*

- installExtension — auto-install Java Pack on clone
- memory — remember "clean already ran"
- runCommand — "Java: Clean Workspace"
- askQuestions — "Which brand to add?"

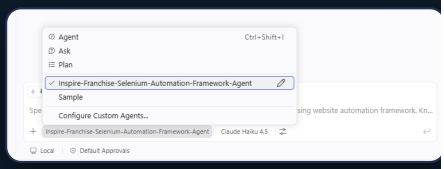
⚡ execute/*

- runInTerminal — .\mvnw.cmd clean test
- runTests — trigger ▶ button per test
- testFailure — read stack traces
- getTerminalOutput — parse PASSED/FAILED
- killTerminal — stop hung Maven builds

📖 read/*

- readFile — read any Java/XML/properties
- problems — see compile errors first
- terminalLastCommand — was clean already run?

Tools declared in the agent YAML frontmatter



✏️ edit/*

- createFile — DunkinPage.java, DunkinTest.java
- editFiles — add case DUNKIN: to factory
- createDirectory — package directories

🔍 search/*

- codebase — "where is cookie handled?"
- textSearch — find all text() locators to fix
- usages — who calls isVisibleAfterWait()?
- changes — git diff since last run

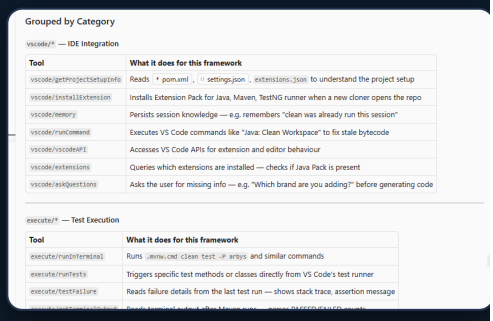
🌐 web/* & agent/*

- web/fetch — inspect live page DOM for new locators
- browser/openBrowserPage — open Extent report
- agent/runSubagent — delegate complex exploration without cluttering chat

Key Sections in the Inspire Franchise Agent

Section	What It Contains	Why It Matters
MANDATORY Rules ×3	`clean test`, cookie banner, 5 XPath rules	**Prevents the most common failures before they happen**
Project Structure	Full file tree with inline ← annotations	Agent edits the right file, never guesses
Brands Table	7 brands with enum, URL slug, properties file	Correct names & paths generated for all brands
Test Case Conventions	TC prefix map, naming pattern, valid groups	Consistent TC IDs without duplicating a registry
Design Patterns & SOLID	9 patterns mapped to exact classes	New code follows the same architecture automatically
Add New Brand (4 Steps)	Copy-paste ready code for each step	New brand in <5 minutes — zero existing file changes
Pitfalls P-01 to P-10	Root cause + fix for every hard-won bug	No developer rediscovers the same issue twice

The Inspire agent in the VS Code agent picker



Section	What It Contains	Why It Matters
Agent Behaviour Rules	10 numbered constraints for code generation	Same quality output from every developer, every time

Demo — Select Agent & Add Baskin Robbins Brand

Part 1 — Selecting the Custom Agent

1. **Open Copilot Chat**
   Press `Ctrl+Shift+I` or click the Copilot icon in the sidebar
2. **Click the Agent Picker (@)**
   Type `@` or click the "Agent" button at the bottom of the chat panel
3. **Select the Inspire Agent**
   Choose `Inspire-Franchise-Selenium-Automation-Framework-Agent` from the dropdown — note the "Workspace" label
4. **Verify Agent is Active**
   The agent name appears in the chat header. All framework rules are now pre-loaded.

Part 2 — Add Baskin Robbins Brand

5. **Type the Request**
   `"Add Baskin Robbins brand to this framework"`
6. **Agent Creates 4 Files Automatically**
   - `BaskinRobbinsPage.java` with correct @FindBy
   - `BrandPageFactory` case BASKIN_ROBBINS
   - `BaskinRobbinsTest.java` extending AbstractBrandTest
   - `testng-baskin-robbins.xml`
7. **Agent Runs Smoke Tests**
   Executes `.\mvnw.cmd clean test -P baskin-robbins -Dgroups=smoke` automatically and verifies all TC-B tests pass for the new brand

Key Takeaways

Speed
Adding a new brand that took a developer a full day now takes 5 minutes with the custom agent.

Consistency
Every developer produces the same quality output — patterns, XPath rules, and pitfall fixes are always applied.

Knowledge Preservation
Every hard-won fix is permanently encoded. No one rediscovers the same bug twice.

The custom agent is not just a productivity tool — it is a living document of your team's collective intelligence.

The .agent.md file lives in .github/agents/, committed to Git — every team member who clones the repo gets the same expert-level Copilot experience immediately.

`Inspire-Franchise-Selenium-Automation-Framework-Agent` `.\mvnw.cmd clean test -P arbys` **Zero existing files modified when adding a brand**